

Data Structures & Algorithms

Chapter 1: Introduction & Sorting Algorithms

1. What is a Data Structure?

A data structure is a way of organizing and storing data in a computer so that it can be accessed and modified efficiently. Different data structures are suited to different kinds of applications. Arrays, linked lists, stacks, queues, trees, and graphs are all examples of data structures. Choosing the right data structure for a problem is one of the most important decisions in software development.

2. Arrays

An array is a collection of elements stored at contiguous memory locations. The idea is to store multiple items of the same type together. Arrays allow random access of elements using an index. The time complexity for accessing an element is $O(1)$, while insertion and deletion at arbitrary positions is $O(n)$. Arrays are used in implementing other data structures like stacks, queues, and heaps.

3. Linked Lists

A linked list is a linear data structure where each element is a separate object called a node. Each node contains data and a reference (pointer) to the next node in the sequence. Unlike arrays, linked lists do not store elements in contiguous memory. Insertion and deletion at the beginning is $O(1)$, but searching is $O(n)$. Types include singly linked list, doubly linked list, and circular linked list.

4. Stacks

A stack is a linear data structure that follows the Last In First Out (LIFO) principle. The element inserted last is the first to be removed. Operations: push (insert), pop (remove), peek (view top element). All operations are $O(1)$. Stacks are used in function call management, undo operations in editors, expression evaluation, and backtracking algorithms.

5. Queues

A queue is a linear data structure that follows the First In First Out (FIFO) principle. Elements are inserted at the rear and removed from the front. Operations: enqueue (insert), dequeue (remove). All operations are $O(1)$. Queues are used in CPU scheduling, disk scheduling, breadth-first search (BFS), and printer spooling.

6. Bubble Sort

Bubble Sort is the simplest sorting algorithm. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. Time Complexity: $O(n^2)$ in worst and average case, $O(n)$ in best case (already sorted). Space Complexity: $O(1)$. It is a stable sorting algorithm. Bubble sort is not suitable for large datasets due to its poor time complexity.

7. Selection Sort

Selection Sort divides the array into a sorted and unsorted portion. It repeatedly finds the minimum element from the unsorted portion and moves it to the beginning. Time Complexity: $O(n^2)$ in all cases. Space Complexity: $O(1)$. Unlike Bubble Sort, Selection Sort makes at most $n-1$ swaps, making it useful when memory writes are costly. It is not a stable sort.

8. Insertion Sort

Insertion Sort builds the sorted array one element at a time by picking each element and placing it in its correct position. Time Complexity: $O(n^2)$ worst case, $O(n)$ best case. Space Complexity: $O(1)$. It is efficient for small datasets and nearly sorted arrays. It is a stable sort and is used as a subroutine in advanced algorithms like Tim Sort and Shell Sort.

9. Merge Sort

Merge Sort is a divide and conquer algorithm. It divides the array into two halves, recursively sorts them, and merges the sorted halves. Time Complexity: $O(n \log n)$ in all cases. Space Complexity: $O(n)$. It is a stable sort and is preferred for sorting linked lists. Merge sort guarantees $O(n \log n)$ performance making it reliable for large datasets.

10. Quick Sort

Quick Sort is a divide and conquer algorithm that picks a pivot element and partitions the array around it. Elements smaller than pivot go left, larger go right. Time Complexity: $O(n \log n)$ average, $O(n^2)$ worst case. Space Complexity: $O(\log n)$. It is the fastest sorting algorithm in practice for most inputs. The choice of pivot affects performance significantly.

11. Binary Search

Binary Search is an efficient algorithm for finding an element in a sorted array. It repeatedly divides the search interval in half. If the target is less than the middle element, search the left half; otherwise search the right half. Time Complexity: $O(\log n)$. Space Complexity: $O(1)$ iterative, $O(\log n)$ recursive. Binary search requires the array to be sorted beforehand.

12. Time and Space Complexity

Time complexity describes how the runtime of an algorithm grows with input size. Space complexity describes memory usage. Big O notation is used: $O(1)$ constant, $O(\log n)$ logarithmic, $O(n)$ linear, $O(n \log n)$ linearithmic, $O(n^2)$ quadratic, $O(2^n)$ exponential. Always aim for the lowest complexity possible. Trade-offs between time and space are common in algorithm design.